ROOTCAUSE SERVER

16 · Glosario

Documento: 16-glossary.md **Versión analizada:** 0.2.0 **Commit:** 767a67b **Fecha del análisis:** 2026-08-26

CONTENIDO

- Conceptos del producto
- Términos de seguridad
- Términos técnicos generales
- Roles de activo
- Estados
  - De un incidente
  - De un activo
  - De un paso de runbook
- Nombres históricos

> Términos técnicos, siglas, conceptos del dominio, roles y estados, explicados para que los entienda alguien sin formación técnica. Cuando un término tiene un significado **específico dentro de RootCause**, se dice.

Conceptos del producto

Activo *(asset)* — Un servidor administrado. Cada equipo con un agente instalado es un activo. En la base de datos es una fila de la tabla `assets`.

Agente *(rootcause-agent)* — El pequeño programa que se instala en cada servidor. Solo observa: no bloquea, no edita configuraciones y solo ejecuta comandos de una lista cerrada de solo lectura. Envía lo que ve al plano de control.

Plano de control *(rootcause-server)* — El programa central. Recibe lo que envían los agentes, aplica las reglas, guarda la evidencia y muestra la consola. Hay uno; los agentes son muchos.

Consola — La página web que muestra el estado. Va **dentro** del programa del servidor, no es un sitio aparte.

Telemetría — El paquete de datos que un agente envía en cada ciclo: uso de recursos y, opcionalmente, la superficie de seguridad.

Sobre *(envelope)* — El nombre técnico de ese paquete. En el código, `TelemetryEnvelope`.

Ciclo — Una vuelta completa del agente: mirar, empaquetar y enviar. Por omisión cada 30 segundos.

Hallazgo — Algo que merece atención, detectado por una regla.

Incidente — Un hallazgo ya guardado, con su evidencia, su causa y su historia. Si la misma condición se repite, **no** se crea un incidente nuevo: se incrementa el contador de ocurrencias del que ya existe.

Evidencia — La observación concreta que sostiene un hallazgo. No es la conclusión, es el dato: «PostgreSQL escucha en `0.0.0.0:5432`, proceso `postgres` con PID 1204».

Causa raíz *(root cause)* — La frase que explica **por qué** ocurre el hallazgo, no solo qué se observó. Da nombre al producto.

Confianza *(confidence)* — Un número de 0 a 1 que dice **cuán seguro** está el sistema de su interpretación. Es un campo separado de la severidad: un hallazgo puede ser muy grave y a la vez incierto. La regla de puertos expuestos tiene confianza 0.99 —el puerto está o no está—; la de salida anómala tiene 0.55, y su propio texto explica que un respaldo programado la explicaría igual de bien.

Severidad — Cuánto importa: informativo, bajo, medio, alto o crítico.

Categoría — A qué familia pertenece un hallazgo. Hay seis: superficie expuesta, intrusión, integridad, disponibilidad, higiene y recursos.

Huella *(fingerprint)* — La etiqueta que identifica un hallazgo de forma única. Sirve para reconocer que «esto ya lo vi» y no duplicarlo. Se compone del identificador del equipo, la regla y, si aplica, el detalle concreto (por ejemplo el puerto).

Postura *(posture)* — Una nota de 0 a 100 que resume cuán defendido está algo, con su letra de la A a la F. **Siempre viaja acompañada de la lista de cosas que no se pudieron revisar**, para que un número alto sobre un servidor sin inspeccionar no se lea como un certificado de salud.

Brecha de recolección *(collection gap)* — Cuando el agente **no pudo** revisar algo. No es un error del sistema: es información. La diferencia entre «no hay puertos abiertos» y «no pude mirar los puertos» es el corazón del producto.

Superficie — Todo aquello a lo que se puede llegar desde fuera. La «superficie expuesta» de un servidor son sus servicios alcanzables.

Alcance *(scope)* — Hasta dónde llega un servicio. Tres valores: **solo local** (nadie fuera del propio equipo), **red interna** (la oficina o el centro de datos) y **público** (cualquiera). *La misma base de datos es rutina en el primer caso y emergencia en el tercero.*

Runbook — La lista de pasos concretos para responder a un hallazgo, con los comandos exactos y cómo deshacerlos. **RootCause lo escribe; nunca lo ejecuta.** Esa decisión es siempre de una persona.

Política de detección — El conjunto de números que decide cuándo saltan las reglas: a qué porcentaje de CPU, tras cuántos intentos fallidos, con cuántos usuarios distintos. Se puede versionar junto al despliegue, y el servidor la valida antes de aceptarla.

Regla — Una pregunta concreta que el sistema le hace a los datos, del tipo *«¿un origen que fallaba repetidamente consiguió entrar?»*. Hay 18, y el propio programa las publica.

Perímetro (en RootCause) — La defensa que el plano de control aplica **a sí mismo**: cuántas peticiones acepta por dirección y cuándo bloquea a quien falla al autenticarse repetidamente.

Términos de seguridad

Fuerza bruta *(brute force)* — Probar contraseñas una tras otra hasta acertar.

Barrido de usuarios *(password spraying)* — Variante del anterior: en vez de muchas contraseñas para un usuario, se prueba la misma contraseña contra muchos usuarios. *No es alguien que olvidó su clave: es una lista.*

Exfiltración — Sacar datos de un sistema sin autorización. La regla de salida anómala busca señales de esto, y dice explícitamente que no puede distinguirla de un respaldo.

Superficie de ataque — Todo lo que alguien de fuera puede tocar. Cuanto menor, mejor.

Endurecimiento *(hardening)* — Configurar algo para que sea más difícil de atacar: cerrar lo que no se usa, quitar permisos que no hacen falta, activar filtros.

Loopback — La dirección `127.0.0.1` (o `::1`), que significa «este mismo equipo». Un servicio en loopback es inalcanzable desde la red.

RFC1918 — El estándar que define los rangos de direcciones privadas: `10.x`, `172.16–31.x`, `192.168.x`. Son direcciones de red interna.

Token — Una cadena secreta larga que hace de contraseña de máquina. Quien la tiene, entra.

Bearer *(portador)* — La forma de enviar un token en una petición web: `Authorization: Bearer <token>`. Se llama «portador» porque quien lo lleva es tratado como legítimo, sin más preguntas.

Comparación en tiempo constante — Comparar dos secretos de forma que tarde **lo mismo** acierten o no. Si tardara distinto, un atacante podría deducir letra a letra la respuesta correta midiendo el tiempo.

Límite de tasa *(rate limiting)* — Poner un tope de peticiones por minuto y por origen.

Cubeta de tokens *(token bucket)* — La técnica concreta que se usa aquí. Imagina un cubo con 600 fichas que se rellena poco a poco; cada petición gasta una ficha. Si el cubo se vacía, hay que esperar. Permite ráfagas cortas sin castigar el uso normal.

Bloqueo *(lockout)* — Dejar fuera temporalmente a una dirección que falló demasiadas veces al autenticarse.

CSP *(Content Security Policy)* — Una instrucción que el servidor le da al navegador diciendo de dónde puede cargar cosas. Aquí es muy estricta: **solo de sí mismo**, y sin ejecutar código escrito dentro del HTML.

XSS *(Cross-Site Scripting)* — Ataque en el que se cuela código malicioso en una página. RootCause se defiende en dos capas: la CSP, y construir cada elemento de la página con funciones en vez de pegando texto.

CSRF *(Cross-Site Request Forgery)* — Ataque que aprovecha la sesión de un usuario desde otro sitio. Aquí no aplica, porque no se usan cookies.

HSTS *(HTTP Strict Transport Security)* — Instrucción al navegador para que solo use conexiones cifradas con ese sitio.

TLS — El cifrado de las conexiones web (la «s» de HTTPS).

Proxy inverso — Un servidor que se pone delante, recibe las conexiones cifradas y las pasa al programa real. Caddy, Nginx y Traefik son ejemplos.

SHA-256 — Una función que convierte cualquier archivo en una cadena corta y única, su «huella». Sirve para saber si algo cambió, **sin revelar su contenido**.

MITRE ATT&CK — Un catálogo público y estándar de técnicas de ataque, con códigos como `T1110` (fuerza bruta). Cada regla de RootCause dice a qué técnicas corresponde, para poder relacionarla con otras herramientas.

SBOM *(Software Bill of Materials)* — La lista completa de las piezas de terceros que lleva un programa dentro. Cuando aparece una vulnerabilidad, es lo que permite saber si te afecta.

CVE / RUSTSEC — Identificadores de vulnerabilidades conocidas. `RUSTSEC` es el registro específico del ecosistema de Rust.

Falso positivo — Una alerta que resulta no ser un problema. El repositorio tiene una plantilla de incidencia dedicada a reportarlos.

Términos técnicos generales

Rust — El lenguaje de programación en el que está escrito todo. Se eligió por seguridad de memoria y por compilar a un binario único en las tres plataformas.

Crate — Un paquete de Rust. Este proyecto tiene tres: `rootcause-core` (el dominio), `rootcause-agent` (el sensor) y `rootcause-server` (el plano de control).

Workspace — El conjunto de crates que se compilan y versionan juntos.

`unsafe` — Una palabra clave de Rust que desactiva algunas comprobaciones de seguridad. En este proyecto **está prohibida**: el código ni siquiera compila si alguien la usa.

Clippy — El analizador de código de Rust. Aquí cada advertencia suya se trata como defecto, no como sugerencia.

MSRV *(Minimum Supported Rust Version)* — La versión más antigua de Rust con la que el proyecto compila. Aquí, `1.95.0`, y CI comprueba que la versión declarada es exactamente la que se compila.

SQLite — Una base de datos que es un solo archivo, sin servidor que instalar.

WAL *(Write-Ahead Logging)* — Un modo de SQLite que permite leer mientras se escribe.

Migración — Un script que cambia la estructura de la base de datos. Aquí se aplican solas al arrancar.

Determinista — Que con la misma entrada produce siempre la misma salida. El motor de detección lo es, y por eso un incidente se puede reproducir meses después.

Idempotente — Que repetir la operación no cambia el resultado. Registrar un activo dos veces deja el mismo estado que registrarlo una.

NDJSON — Un archivo donde cada línea es un documento JSON independiente. Es el formato del paquete de evidencia: se puede procesar línea a línea sin cargarlo entero.

Prometheus — Un sistema muy extendido para recoger métricas y lanzar alertas.

CI/CD — Integración y entrega continuas: la automatización que compila, prueba y publica en cada cambio.

Pinnear a SHA — Fijar una dependencia externa a una versión exacta e inmutable, en vez de a una etiqueta que alguien podría mover. Aquí es obligatorio para toda acción de GitHub, y un guardián lo comprueba.

ADR *(Architecture Decision Record)* — Un documento corto que registra una decisión de arquitectura y **por qué** se tomó. Este proyecto tiene tres.

Roles de activo

Rol	Etiqueta	Qué significa
Servidor de borde	`role=edge`	Está en Internet a propósito. Puede publicar 80 y 443 sin generar hallazgo
Servidor interno	`role=internal`	**Valor por omisión.** Solo debería alcanzarse desde la red interna
Servidor de base de datos	`role=database`	Nunca debería responder a Internet. Cualquier puerto conocido alcanzable públicamente es crítico

Rol	Etiqueta	Qué significa
Estación de trabajo	`role=workstation`	Equipo de una persona

Estados

De un incidente

Estado	Significado
`open` (abierto)	Sin atender
`acknowledged` (reconocido)	Alguien lo vio y se hace cargo
`resolved` (resuelto)	Cerrado, a mano o porque la condición dejó de observarse

Reconocer no es arreglar. Si la condición sigue ocurriendo, el incidente vuelve a `open` por sí solo.

De un activo

Estado	Cuándo
`online` (en línea)	Reportó hace menos de 120 segundos
`stale` (rezagado)	Lleva entre 120 y 600 segundos sin reportar
`offline` (fuera de línea)	Lleva más de 600 segundos sin reportar

De un paso de runbook

Tipo	Qué hace
`inspect` (inspeccionar)	Solo mira. No cambia nada
`contain` (contener)	Limita el alcance o bloquea un origen. **Reversible**
`remediate` (corregir)	Cambia configuración. Reversible, pero conviene revisarlo antes

Nombres históricos

RootCause — La familia de herramientas de la que este proyecto forma parte, junto a los inspectores de Windows, macOS, web, móvil, QR, Bitcoin y blockchain. Todos comparten la misma idea: explicar la causa con evidencia, en vez de limitarse a avisar.

`rca.rs` — Un módulo que contenía el motor de detección antiguo. Dejó de estar conectado al programa cuando la detección pasó a la carpeta `detect/`, y se eliminó en el commit `2efedb2` precisamente porque un archivo con reglas viejas conviviendo con las nuevas es el tipo de cosa que alguien lee después y toma por la implementación vigente.

Protocolo 1.0 y 1.1 — Las dos versiones del formato de comunicación entre agente y servidor. La 1.1 añadió la superficie de seguridad; como el campo es opcional, un agente 1.0 sigue funcionando contra un servidor 1.1.

Anterior: 15 · Riesgos y deuda técnica · **Siguiente:** 17 · Resumen ejecutivo · **Índice:** README